

프로그래밍 언어 : 개발에 사용하는 언어

9장. JavaScript 기초: 변수, 자료형, 제어문

HTML 파일 안에서
학습 목표 - script 태그와 콘솔로 JavaScript를 실행하고 결과를 확인할 수 있습니다 -
const와 let으로 적절히 변수를 선언할 수 있습니다 - 자료형과 타입 변환, ===의 의미를 설명할 수 있습니다 - 조건문과 반복문으로 프로그램의 흐름을 제어할 수 있습니다

9.1 JavaScript를 페이지에 연결하기

도입

지금까지 HTML로 구조를 만들고 CSS로 스타일을 입혔습니다. 그런데 페이지가 아무리 예쁘더라도, 버튼을 눌러도 아무 일이 일어나지 않으면 '살아있는 웹'이라고 할 수 없습니다.

JavaScript(자바스크립트)는 바로 그 '살아있는 동작'을 담당합니다. 이 절에서는 JavaScript 코드를 HTML 페이지에 연결하는 방법과, 결과를 확인하는 핵심 도구인 콘솔(Console)을 살펴봅니다.

핵심 개념 설명

script 태그로 JS 불러오기

JavaScript를 HTML 페이지에 포함하는 방법은 두 가지입니다. 첫 번째는 `<script>` 태그 사이에 코드를 직접 쓰는 **인라인 방식**이고, 두 번째는 별도의 `.js` 파일을 만들어 `src` 속성으로 불러오는 **외부 파일 방식**입니다. 실무에서는 관심사 분리를 위해 외부 파일 방식을 권장합니다.

(수업 기준)
과거

현재

브라우저 내 실행되는 언어
(client)

브라우저 단독 실행 가능
: Node.js 실행하는
실행기

`<script src="경로" defer></script>`

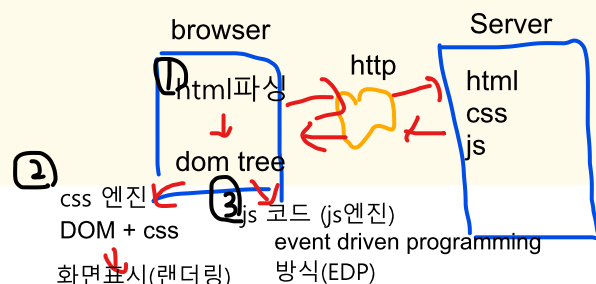
이벤트 소스
(DOM으로 관리)

button

클릭이라는
이벤트

EDP 3가지 핵심요소

- Event : 클릭, 터치, 다운로드 등
- Event Listener/Observer
- Event Handler: 이벤트 발생 시 실행 코드.반복가능성 높.
- > 함수로 제작(Callback Function)



항목	내용
설명	외부 JavaScript 파일을 HTML 문서에 연결합니다
주요 속성	<code>src</code> (string, 필수): JS 파일 경로
	<code>defer</code> (boolean, 선택): HTML 파싱 완료 후 실행
	<code>async</code> (boolean, 선택): 파싱과 병렬 다운로드 후 즉시 실행
권장 위치	<code></body></code> 바로 위 또는 <code><head></code> 안에 <code>defer</code> 함께 사용
사용 예시	<code><script src="app.js" defer></script></code>

defer로 안전하게 로드하기

`<script>` 태그를 `<head>` 안에 두면 브라우저가 HTML을 파싱하다가 스크립트를 먼저 내려받고 실행합니다. 이 시점에는 `<body>` 안의 요소들이 아직 만들어지지 않았으므로, JavaScript로 요소를 찾으려 하면 `null` 을 반환합니다. `defer` 속성을 추가하면 HTML 파싱이 완전히 끝난 뒤 스크립트를 실행하여 이 문제를 예방합니다.

console.log로 결과 확인하기

`console.log()` 는 개발자 도구의 콘솔(Console) 패널에 값을 출력합니다. 화면에 보이지 않아도 코드가 제대로 동작하는지 확인할 수 있는 가장 기본적인 디버깅 도구입니다.

`console.log(value1, value2, ... )`

항목	내용
설명	브라우저 개발자 도구 콘솔에 값을 출력합니다
매개변수	<code>value</code> (any): 출력할 값. 여러 개를 쉼표로 구분하면 공백을 두고 출력
반환값	<code>undefined</code>
사용 예시	<code>console.log('이름:', name)</code>

개발자 도구 콘솔 사용법

크롬(Chrome)/엣지(Edge) 브라우저에서 **F12** 또는 **Ctrl+Shift+I** (Mac: **Cmd+Option+I**)를 누르면 개발자 도구가 열립니다. 상단 탭 중 **Console** 탭을 선택하면 **console.log** 출력, 에러 메시지, 그리고 직접 코드를 입력해 즉시 실행하는 모든 작업을 할 수 있습니다.

코드로 확인하기

script 태그를 **defer** 와 함께 **<head>** 에 배치하고, 콘솔로 첫 번째 메시지를 출력하는 기본 패턴을 보여줍니다.



```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>JavaScript 첫 번째 연결</title>
  <!-- defer: HTML 파싱 완료 후 실행 -->
  <script src="app.js" defer></script>
</head>
<body>
  <h1>콘솔을 확인해 보세요</h1>
  <p id="message">아직 아무것도 없습니다.</p>
</body>
</html>
```

원래는 아직 <body>는 존재 x.
defer 속성 주면
html 파싱 완전히 끝난 뒤 js 실행

app.js 파일 내용:

```
// 콘솔에 메시지 출력
console.log('JavaScript 연결 성공!');

// 페이지 요소를 찾아 내용 변경
const para = document.getElementById('message');
para.textContent = 'JavaScript가 실행되었습니다!';
```

브라우저에서 파일을 열고 개발자 도구 콘솔을 확인하면 다음과 같이 출력됩니다:

```
// 콘솔 출력:  
JavaScript 연결 성공!
```

줄별 코드 설명

줄	코드	설명
2	<code>console.log('JavaScript 연결 성...</code>	콘솔에 문자열을 출력합니다. 코드가 실행되었는지 확인하는 가장 기본적인 방법입니다
5	<code>const para = document.getEleme...</code>	id가 <code>message</code> 인 요소를 찾아 변수에 저장합니다
6	<code>para.textContent = ' ... '</code>	요소의 텍스트 내용을 새 문자열로 교체합니다

인라인 방식(한 파일로 실습할 경우)은 다음과 같습니다:

```
<!DOCTYPE html>  
<html lang="ko">  
<head>  
  <meta charset="UTF-8">  
  <title>인라인 스크립트 예제</title>  
</head>  
<body>  
  <h1>인라인 스크립트</h1>  
  <!-- body 끝 직전에 script를 두면 요소가 모두 준비된 뒤 실행 -->  
  <script>  
    console.log('안녕하세요, JavaScript!');  
    console.log('현재 URL:', window.location.href);  
  </script>  
</body>  
</html>
```

```
// 콘솔 출력:  
안녕하세요, JavaScript!  
현재 URL: file:///C:/projects/index.html
```

팁: 실습할 때는 `<script>` 태그를 `</body>` 바로 위에 두는 방식도 동작합니다. 다만 파일이 여러 개로 늘어나면 `defer`를 사용하는 외부 파일 방식이 더 깔끔하고 유지보수에 유리합니다.

주의: `<head>` 안에 `defer` 없이 `<script src="..."></script>`를 두면, 아직 만들어진 요소 조작하려다 에러가 발생할 수 있습니다. `defer`는 습관처럼 붙이는 것이 안전합니다.

절 요약

- JavaScript는 `<script>` 태그로 HTML과 연결합니다.
- 외부 파일 방식이 인라인 방식보다 유지보수에 유리합니다.
- `defer` 속성은 HTML 파싱 완료 후 스크립트를 실행하여 요소를 안전하게 접근하게 합니다.
- `console.log()`는 개발자 도구 콘솔에 값을 출력하는 핵심 디버깅 도구입니다.

9.2 변수와 상수

도입

프로그램은 데이터를 기억하고 처리해야 합니다. **변수(Variable)**는 값에 이름을 붙여 저장하는 공간입니다. JavaScript에는 변수를 선언하는 키워드로 `const`, `let`, `var` 세 가지가 있지만, 현대 JavaScript에서는 `const`와 `let`만 사용하고 `var`는 쓰지 않습니다. 왜 그런지 스코프와 호이스팅 개념을 통해 알아봅시다.

핵심 개념 설명

저장공간 생성 변수와 상수 (const / let)

const (상수)는 한 번 값을 할당하면 재할당이 불가능한 변수입니다. **let** (변수)은 나중에 값을 바꿀 수 있는 변수입니다.

베스트 프랙티스: 기본적으로 **const** 를 사용하고, 나중에 값을 바꿔야 하는 경우에만 **let** 으로 바꾸세요. 재할당이 필요 없는 경우가 훨씬 많으므로, **const** 를 기본값으로 삼으면 의도치 않은 값 변경을 방지할 수 있습니다.

const와 let의 차이

파이썬에서는 로컬변수라고 부름

const 와 **let** 모두 블록 스코프(Block Scope)를 가집니다. 블록 **{ }** 안에서 선언된 변수는 해당 블록 안에서만 접근할 수 있습니다.

var를 쓰지 않는 이유

var 는 블록 스코프가 아닌 함수 스코프(Function Scope)를 가집니다. **if** 나 **for** 블록 안에서 선언해도 블록 밖에서 접근됩니다. 또한 **var** 로 선언된 변수는 호이스팅(Hoisting) 때문에 초기화 전에 선언 전에도 **undefined** 로 접근이 가능합니다. 이런 동작은 예상치 못한 버그를 만들기 쉽습니다.

정의되지 않은. null과 다름. null은 "없다."
엔진이 지정(cf> null 은 개발자가 지정)

블록 스코프와 호이스팅 맛보기

호이스팅(Hoisting)이란 변수와 함수 선언이 코드 실행 전에 해당 스코프의 최상단으로 끌어 올려지는 동작입니다. **var** 는 선언이 호이스팅되어 **undefined** 로 초기화되지만, **let** 과 **const** 는 호이스팅되더라도 초기화되지 않아 선언 전에 접근하면 에러가 발생합니다. 이를 일시적 사각지대(TDZ, Temporal Dead Zone)라 합니다.

변수 명명 규칙

- 영문자, 숫자, **_**, **\$** 만 사용합니다 (첫 글자는 숫자 불가).
- 의미 있는 이름을 사용합니다 (**x** 보다 **userName** 이 낫습니다).
- 여러 단어는 카멜케이스(camelCase)로 씁니다 (**myUserName**).
- 예약어(**let** , **const** , **if** 등)는 사용할 수 없습니다.

코드로 확인하기

`const` 와 `let` 의 기본 사용법과 `var` 의 문제점을 비교합니다.

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>변수와 상수</title>
</head>
<body>
<script>
  // const: 재할당 불가
  const siteName = '나의 블로그';
  console.log(siteName);      // 나의 블로그

  // 아래 줄의 주석을 해제하면 에러 발생
  // siteName = '다른 블로그'; // TypeError: Assignment to constant variable.

  // let: 재할당 가능
  let score = 80;
  console.log('초기 점수:', score);
  score = 95;                // 재할당 가능
  console.log('수정 점수:', score);

  // 블록 스코프 확인
  {
    const blockVar = '블록 안에서만 유효';
    console.log(blockVar);    // 블록 안에서만 유효
  }
  // console.log(blockVar);    // ReferenceError: blockVar is not defined

  // var의 문제: 블록을 벗어나도 접근 가능
  if (true) {
    var leaky = 'var는 블록을 탈출한다';
  }
  console.log(leaky);        // var는 블록을 탈출한다 (버그 위험)
</script>
</body>
</html>
```

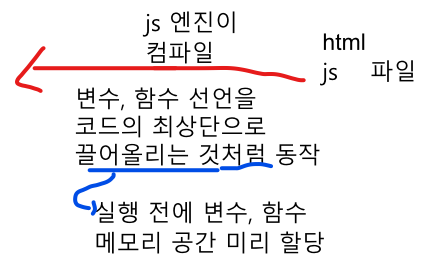
```
// 콘솔 출력:
나의 블로그
초기 점수: 80
수정 점수: 95
블록 안에서만 유효
var는 블록을 탈출한다
```

줄별 코드 설명

줄	코드	설명
3	<code>const siteName = '나의 블로그'</code>	문자열 값을 상수에 저장합니다. 이후 재할당하면 <code>TypeError</code> 가 발생합니다
8	<code>let score = 80</code>	숫자 값을 변수에 저장합니다. 이후 <code>= 95</code> 로 재할당이 가능합니다
13	<code>{ const blockVar = '...' }</code>	블록 <code>{ }</code> 안에서 선언된 변수는 블록 밖에서 접근할 수 없습니다
20	<code>var leaky = '...'</code>	<code>var</code> 는 블록 스코프를 무시하므로 <code>if</code> 블록 밖에서도 접근됩니다

호이스팅 동작을 콘솔에서 직접 확인하는 예제입니다.

호이스팅



예. var로 선언 => undefined => var 쓰지 말자.
let, const => TDZ(임시사각지대)

변수 선언 |-----> 초기화

let, const 초기화되기 전에는 못 씀.


```

<!DOCTYPE html>
<html lang="ko">
<head><meta charset="UTF-8"><title>호이스팅</title></head>
<body>
<script>
  // var 호이스팅: 선언 전 접근해도 undefined 반환
  console.log(hoistedVar); // undefined (에러 없음)
  var hoistedVar = '나중에 선언';

  // let 호이스팅: 선언 전 접근하면 ReferenceError
  try {
    console.log(hoistedLet); // ReferenceError 발생
  } catch (e) {
    console.log('에러:', e.message); // 에러: Cannot access 'hoistedLet' before
  }
  let hoistedLet = '안전한 선언';
</script>
</body>
</html>

```

```

// 콘솔 출력:
undefined
에러: Cannot access 'hoistedLet' before initialization

```

주의: `const` 로 선언한 객체나 배열의 내부 값은 변경할 수 있습니다. `const` 가 막는 것은 변수 자체의 재할당이지, 객체의 프로퍼티 변경이 아닙니다. 예: `const user = { name: '철수' }; user.name = '영희';` — 이것은 정상 동작합니다.

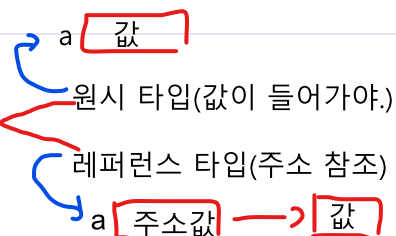
비교 테이블

구분	const	let	var
재할당	불가	가능	가능
스코프	블록	블록	함수
호이스팅 시 초기화	미초기화(TDZ)	미초기화(TDZ)	undefined
선언 전 접근	ReferenceError	ReferenceError	undefined
권장 여부	기본값으로 사용	재할당 필요 시	사용 금지

절 요약

- `const` 는 재할당 불가, `let` 은 재할당 가능한 블록 스코프 변수입니다.
- `var` 는 함수 스코프와 호이스팅 문제로 현대 JavaScript에서 사용하지 않습니다.
- 기본적으로 `const` 를 사용하고, 값을 바꿔야 할 때만 `let` 으로 바꿉니다.
- 변수는 의미 있는 카멜케이스 이름으로 선언합니다.

9.3 자료형 이해하기



<--> 파이썬: 데이터 타입 1개(object)

도입

JavaScript의 모든 값은 특정 **자료형(Data Type)**을 가집니다. 자료형에 따라 사용할 수 있는 연산이 달라지므로, 각 타입의 특성을 이해하는 것이 버그 없는 코드를 작성하는 첫걸음입니다. 특히 `null` 과 `undefined` 의 미묘한 차이, 그리고 JavaScript의 **암묵적 타입 변환**은 중급 개발자가 반드시 짚고 넘어가야 할 부분입니다.

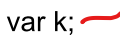
명시적: 개발자가

암묵적: 엔진이

핵심 개념 설명

원시 타입 (Primitive Types)

JavaScript의 **원시 타입(Primitive Type)**은 **값 자체를 직접 저장하는 불변(immutable) 타입**입니다. 주요 원시 타입은 다음과 같습니다:

- **number** (숫자): 정수와 실수를 모두 포함합니다. 특수값으로 `Infinity`, `-Infinity`, `NaN` (Not a Number)이 있습니다.
- **string** (문자열): 텍스트 데이터입니다. 작은따옴표 `'`, 큰따옴표 `"`, 백틱 ``` 으로 표현합니다.
- **boolean** (불리언): 참(`true`) 또는 거짓(`false`)만 가능한 논리 값입니다.
- **bigint**: 매우 큰 정수를 다룰 때 사용합니다. 숫자 뒤에 `n` 을 붙입니다.
- **symbol**: 유일한 식별자를 만들 때 사용합니다.
- **undefined**: 변수가 선언되었지만 값이 할당되지 않은 상태입니다.
 `var k;`  undefined로 초기화
- **null**: 의도적으로 '값이 없음'을 나타내는 상태입니다.
 `let k;`  초기화x
 `const k;`

이 외에 **object** (객체)가 있으며, 배열과 함수도 내부적으로 객체에 해당합니다.

 선언: function 키워드
표현식: () 사용

null과 undefined의 차이

undefined 는 값이 아직 없는 상태입니다. 변수를 선언만 하고 초기화하지 않으면 자동으로 **undefined** 가 됩니다. **null** 은 개발자가 의도적으로 비어있음을 표현할 때 사용합니다. "아직 정해지지 않았다"는 **undefined**, "없다고 명시하겠다"는 **null** 로 이해하면 됩니다.

typeof로 타입 확인하기

typeof 연산자는 값의 타입을 문자열로 반환합니다. 단, **null** 에 대해 **"object"** 를 반환하는 것은 JavaScript 초기 설계의 버그로, 현재는 사양으로 굳어진 **역사적 오류**입니다.

암묵적·명시적 타입 변환

JavaScript는 **동적 타입(Dynamic Typing)** 언어입니다. 연산자를 사용할 때 타입이 맞지 않으면 엔진이 자동으로 타입을 변환(암묵적 타입 변환, Type Coercion)합니다. 이 동작은 예상치 못한 결과를 낳을 수 있어 주의가 필요합니다.

권장 **명시적 타입 변환**은 `Number()`, `String()`, `Boolean()` 함수를 직접 호출하여 개발자가 의도적으로 타입을 바꾸는 방법입니다.

코드로 확인하기

7가지 원시 타입을 선언하고 **typeof** 로 확인합니다.

```

<!DOCTYPE html>
<html lang="ko">
<head><meta charset="UTF-8"><title>자료형</title></head>
<body>
<script>
  // 원시 타입 선언
  const num = 42;
  const pi = 3.14;
  const text = '안녕하세요';
  const template = `현재 숫자: ${num}`; // 템플릿 리터럴
  const isLoggedIn = true;
  const nothing = null;
  let notAssigned; // undefined

  // typeof로 타입 확인
  console.log(typeof num); // number
  console.log(typeof text); // string
  console.log(typeof isLoggedIn); // boolean
  console.log(typeof nothing); // object ← 역사적 버그!
  console.log(typeof notAssigned); // undefined
  console.log(typeof 9007199254740999n); // bigint

  // 특수 숫자값
  console.log(1 / 0); // Infinity
  console.log(-1 / 0); // -Infinity
  console.log(0 / 0); // NaN
  console.log(typeof NaN); // number (NaN도 숫자 타입)
</script>
</body>
</html>

```

해당 코드가
밑에 있었으면
에러

```

// 콘솔 출력:
number
string
boolean
object
undefined
bigint
Infinity
-Infinity
NaN
number

```

줄별 코드 설명

줄	코드	설명
3	<code>const num = 42</code>	정수 값을 상수에 저장합니다
5	<code>const template = \ 현재 숫자: \${...}</code>	
15	<code>typeof nothing</code>	<code>null</code> 은 <code>"object"</code> 를 반환합니다. JavaScript의 역사적 버그이므로 기억해 두어야 합니다
22	<code>0 / 0</code>	0을 0으로 나누면 <code>NaN</code> (Not a Number)이 됩니다. <code>NaN</code> 은 <code>number</code> 타입이지만 유효하지 않은 숫자입니다

암묵적 타입 변환의 함정과 명시적 타입 변환을 비교합니다.

```
<!DOCTYPE html>
<html lang="ko">
<head><meta charset="UTF-8"><title>타입 변환</title></head>
<body>
<script>
  // 암묵적 타입 변환 - 예상치 못한 결과
  console.log('5' + 3);    // '53' (숫자가 문자열로 변환됨)
  console.log('5' - 3);    // 2 (문자열이 숫자로 변환됨)
  console.log('5' * '3');  // 15 (두 문자열 모두 숫자로 변환)
  console.log(true + 1);   // 2 (true → 1 변환)
  console.log(false + 1);  // 1 (false → 0 변환)
  console.log(null + 1);   // 1 (null → 0 변환)
  console.log(undefined + 1); // NaN (undefined는 숫자로 변환 불가)

  // 명시적 타입 변환 - 의도를 코드에 명확히 표현
  console.log(Number('5') + 3); // 8 (의도적 변환)
  console.log(String(42));      // '42'
  console.log(Boolean(0));      // false
  console.log(Boolean(''));     // false (빈 문자열 → false)
  console.log(Boolean('hello')); // true
  console.log(Boolean(null));   // false
</script>
</body>
</html>
```

```
// 콘솔 출력:
'53'
2
15
2
1
1
NaN
8
'42'
false
false
true
false
```

줄별 코드 설명

줄	코드	설명
3	<code>'5' + 3</code>	<code>+</code> 연산자에 문자열이 있으면 나머지 피연산자도 문자열로 변환하여 이어 붙입니다
4	<code>'5' - 3</code>	<code>-</code> 연산자는 항상 숫자 연산을 시도하므로 <code>'5'</code> 를 <code>5</code> 로 변환한 뒤 빼기를 수행합니다
10	<code>Number('5') + 3</code>	<code>Number()</code> 함수로 명시적 변환 후 덧셈을 수행합니다. 의도가 코드에 드러납니다
12	<code>Boolean(0)</code>	숫자 <code>0</code> , 빈 문자열 <code>''</code> , <code>null</code> , <code>undefined</code> , <code>NaN</code> 은 <code>false</code> 로 변환됩니다(falsy 값)

베스트 프랙티스: 문자열로 된 숫자를 연산에 사용하기 전에 `Number()` 또는 `parseInt()`, `parseFloat()` 로 명시적 변환하세요. `+` 단항 연산자(`+'5'`)를 사용하는 방법도 있지만, 가독성을 위해 `Number()` 를 더 권장합니다.

주의: `NaN` 은 자기 자신과도 같지 않습니다. `NaN === NaN` 의 결과는 `false` 입니다. 숫자가 유효한지 확인하려면 `Number.isNaN(value)` 또는 `isNaN(value)` 를 사용하세요.

비교 테이블: Falsy 값과 Truthy 값

값	Boolean 변환 결과	설명
<code>0</code> , <code>-0</code>	<code>false</code>	숫자 0
<code>''</code> , <code>""</code>	<code>false</code>	빈 문자열
<code>null</code>	<code>false</code>	빈 값
<code>undefined</code>	<code>false</code>	미할당
<code>NaN</code>	<code>false</code>	유효하지 않은 숫자
나머지 모든 값	<code>true</code>	<code>"0"</code> , <code>[]</code> , <code>{}</code> 포함

절 요약

- JavaScript의 주요 원시 타입은 `number`, `string`, `boolean`, `null`, `undefined` 입니다.
- `undefined` 는 값이 없는 상태, `null` 은 의도적으로 비어있음을 표현합니다.
- `typeof null` 이 `"object"` 를 반환하는 것은 역사적 버그입니다.
- `+` 연산자는 문자열이 있으면 이어붙임(concatenation)을 수행하여 예상치 못한 결과를 낼 수 있습니다.
- 타입 변환이 필요할 때는 `Number()`, `String()`, `Boolean()` 으로 명시적 변환을 권장합니다.

9.4 연산자와 조건문

도입

프로그램은 데이터를 **계산하고(연산자)** 상황에 따라 **다른 길을 선택(조건문)** 해야 합니다. 이 절에서는 JavaScript의 주요 연산자와, 그 중에서도 가장 중요한 실수 포인트인 `==` vs `===` 를 살펴봅니다. 그리고 `if/else`, `switch`, 삼항 연산자로 프로그램의 흐름을 제어하는 방법을 익힙니다.

핵심 개념 설명

산술 연산자

기본적인 수학 연산을 수행합니다.

연산자	의미	예시
<code>+</code>	덧셈 / 문자열 연결	<code>3 + 4 → 7</code> , <code>'a' + 'b' → 'ab'</code>
<code>-</code>	빼기	<code>7 - 3 → 4</code>
<code>*</code>	곱셈	<code>3 * 4 → 12</code>
<code>/</code>	나눗셈	<code>10 / 3 → 3.333 ...</code>
<code>%</code>	나머지(모듈로)	<code>10 % 3 → 1</code>
<code>**</code>	거듭제곱	<code>2 ** 10 → 1024</code>
<code>++</code>	1 증가	<code>count++</code>
<code>--</code>	1 감소	<code>count--</code>

비교 연산자와 동등(==) vs 일치(===)

`1 == "1" : true`

== (동등 연산자)는 두 값을 비교할 때 타입이 다르면 자동으로 타입 변환(강제 변환, Type Coercion)을 수행한 후 비교합니다. 반면 **=== (일치 연산자)**는 타입 변환 없이 값과 타입을 모두 비교합니다. 예측하기 어려운 **==**의 동작 때문에 JavaScript에서는 항상 **===**를 사용하는 것이 원칙입니다.

논리 연산자

연산자	의미	예시
<code>&&</code>	AND (모두 참이어야 참)	<code>true && false → false</code>
<code> </code>	OR (하나라도 참이면 참)	<code>true false → true</code>
<code>!</code>	NOT (반전)	<code>!true → false</code>
<code>??</code>	널 병합 (Nullish Coalescing)	<code>null ?? '기본값' → '기본값'</code>

?? (널 병합 연산자)는 왼쪽 피연산자가 `null` 또는 `undefined` 일 때만 오른쪽 값을 반환합니다. `||` 는 falsy 값(`0`, `''` 포함) 전부를 오른쪽으로 넘기지만, `??` 는 `null` / `undefined` 만 처리합니다.

if / else 조건문

조건에 따라 다른 코드를 실행합니다. 조건식은 불리언으로 평가되며, truthy/falsy 값도 사용할 수 있습니다.

switch 문

하나의 변수를 여러 값과 비교할 때 `if/else if` 를 나열하는 대신 사용합니다. 각 `case` 끝에 `break` 를 빠뜨리면 다음 `case` 로 실행이 이어지는(**폴스루(fall-through)**) 버그가 발생하므로 주의합니다.

삼항 연산자

`조건 ? 참일 때 값 : 거짓일 때 값` 형식으로 `if/else` 를 한 줄로 표현합니다. 단순한 경우에 적합하며, 중첩하면 가독성이 떨어지므로 자제합니다.

코드로 확인하기

`==` 와 `===` 의 차이를 집중적으로 비교합니다.

```

<!DOCTYPE html>
<html lang="ko">
<head><meta charset="UTF-8"><title>동등 vs 일치</title></head>
<body>
<script>
  // = 동등 연산자: 타입 변환 후 비교 (예측 어려움)
  console.log(0 = false);    // true (false가 0으로 변환)
  console.log(0 = '');       // true (''가 0으로 변환)
  console.log(1 = '1');       // true ('1'이 1로 변환)
  console.log(null = undefined); // true (특수 규칙)

  // === 일치 연산자: 타입+값 모두 같아야 true
  console.log(0 === false);  // false (number ≠ boolean)
  console.log(0 === '');     // false (number ≠ string)
  console.log(1 === '1');     // false (number ≠ string)
  console.log(null === undefined); // false (null ≠ undefined)

  // 실무에서는 항상 === 사용
  const input = '5';          // 폼 입력은 항상 문자열
  if (input === 5) {
    console.log('숫자 5와 일치'); // 실행 안 됨
  } else {
    console.log('문자열 "5"는 숫자 5와 다릅니다'); // 실행됨
  }
</script>
</body>
</html>

```

```

// 콘솔 출력:
true
true
true
true
false
false
false
false
문자열 "5"는 숫자 5와 다릅니다

```

줄별 코드 설명

줄	코드	설명
3	<code>0 = false</code>	<code>false</code> 가 <code>0</code> 으로 변환된 후 <code>0 == 0</code> 이 되어 <code>true</code> 를 반환합니다
5	<code>1 = '1'</code>	문자열 <code>'1'</code> 이 숫자 <code>1</code> 로 변환된 후 <code>1 == 1</code> 이 되어 <code>true</code> 를 반환합니다
8	<code>0 === false</code>	타입 변환 없이 비교하므로 <code>number</code> 와 <code>boolean</code> 이 달라 <code>false</code> 를 반환합니다
14	<code>const input = '5'</code>	HTML 폼에서 받은 값은 항상 문자열 타입입니다. <code>===</code> 로 비교하면 숫자 <code>5</code> 와 다를 것을 정확히 감지합니다

`if/else`, `switch`, 삼항 연산자를 비교하는 실용 예제입니다.

```

<!DOCTYPE html>
<html lang="ko">
<head><meta charset="UTF-8"><title>조건문</title></head>
<body>
<script>
  const score = 78;

  // if / else if / else
  if (score ≥ 90) {
    console.log('등급: A');
  } else if (score ≥ 80) {
    console.log('등급: B');
  } else if (score ≥ 70) {
    console.log('등급: C');
  } else {
    console.log('등급: D');
  }

  // switch 문
  const day = 'MON';
  switch (day) {
    case 'MON':
    case 'TUE':
    case 'WED':
    case 'THU':
    case 'FRI':
      console.log('평일');
      break; // break 필수
    case 'SAT':
    case 'SUN':
      console.log('주말');
      break;
    default:
      console.log('알 수 없는 요일');
  }

  // 삼항 연산자
  const age = 20;
  const status = age ≥ 18 ? '성인' : '미성년';
  console.log(status); // 성인

  // 널 병합 연산자 (??)
  const nickname = null;
  const displayName = nickname ?? '익명 사용자';
  console.log(displayName); // 익명 사용자

  const score2 = 0;
  // || 는 0도 falsy로 처리 (의도와 다를 수 있음)
  console.log(score2 || 100); // 100 (의도치 않음)
  // ?? 는 null/undefined만 처리

```

```

    console.log(score2 ?? 100);    // 0 (올바른 결과)
  </script>
</body>
</html>

```

```

// 콘솔 출력:
등급: C
평일
성인
이름 사용자
100
0

```

줄별 코드 설명

줄	코드	설명
4	<code>if (score ≥ 90)</code>	<code>>=</code> (크거나 같음) 비교 연산자로 조건을 평가합니다
20	<code>case 'MON': case 'TUE':</code>	<code>break</code> 없이 <code>case</code> 를 연속 나열하면 같은 코드 블록을 공유합니다 (의도적 폴스루)
26	<code>break</code>	<code>switch</code> 각 케이스 실행 후 반드시 <code>break</code> 로 탈출합니다. 없으면 다음 케이스로 이어집니다
34	<code>age ≥ 18 ? '성인' : '미성년'</code>	<code>조건 ? 참값 : 거짓값</code> 형태의 삼항 연산자입니다
42	<code>score2 ?? 100</code>	<code>??</code> 는 <code>null</code> / <code>undefined</code> 일 때만 오른쪽 값을 반환합니다. <code>0</code> 은 유효한 값이므로 <code>0</code> 을 반환합니다

베스트 프랙티스: `==` 대신 항상 `===` 을 사용하세요. ESLint 같은 린터 도구를 사용하면 `==` 사용을 자동으로 경고해 줍니다. `||` 대신 기본값 처리에는 `??` 를 사용하면 `0` 이나 빈 문자열을 의도치 않게 무시하는 버그를 방지할 수 있습니다.

절 요약

- 산술 연산자(`+` , `-` , `*` , `/` , `%` , `**`)로 계산을 수행합니다.
- `===` 은 타입과 값 모두 비교, `==` 은 타입 변환 후 비교합니다. 항상 `===` 를 사용합니다.
- `if/else if/else` 로 다중 조건 분기를 표현합니다.
- `switch` 에는 반드시 `break` 를 붙여 폴스루(fall-through) 버그를 방지합니다.
- `??` (널 병합)는 `null` / `undefined` 에만 기본값을 적용합니다.

9.5 반복문으로 작업 자동화하기

도입

같은 작업을 100번 반복해야 한다면 코드를 100번 쓸 수는 없습니다. **반복문(Loop)**은 조건이 충족되는 동안 코드를 자동으로 반복 실행합니다. 컨베이어 벨트처럼 동일한 작업을 지정한 횟수나 조건만큼 처리합니다. `for` , `while` , `for ... of` 의 차이와 `break` , `continue` 를 이용한 흐름 제어를 배웁니다.

핵심 개념 설명

for 문 기본 구조

`for (초기화; 조건; 증감) { 본문 }` 형태입니다. **초기화**는 반복 시작 전 한 번만 실행되고, 조건이 `true` 인 동안 **본문**을 반복하며, 매 반복 후 **증감**을 실행합니다.

for 반복문 실행 흐름 순서도

while과 do...while

`while` 문은 조건을 먼저 평가한 뒤 참이면 본문을 실행합니다. **반복 횟수를 미리 알 수 없을 때 적합**합니다. `do ... while` 문은 본문을 먼저 실행한 후 조건을 평가하므로, 최소 한 번은 반드시 실행됩니다.

주의: `while` 문을 사용할 때 조건이 절대 `false` 가 되지 않으면 **무한 루프(Infinite Loop)**에 빠집니다. 반드시 조건이 `false` 로 바뀌는 시점이 있어야 합니다. 브라우저

가 응답을 멈추면 탭을 강제 종료해야 합니다.

for...of로 순회하기

`for ... of` 는 배열, 문자열처럼 반복 가능한(iterable) 값을 순서대로 꺼내어 처리합니다. 인덱스 관리가 필요 없어 배열 순회에 더 간결합니다.

break·continue로 흐름 제어하기

- **break** : 반복문을 즉시 종료합니다.
- **continue** : 현재 반복을 건너뛰고 다음 반복으로 넘어갑니다.

코드로 확인하기

`for` 문의 기본 구조와 누적 계산을 보여줍니다.

```
<!DOCTYPE html>
<html lang="ko">
<head><meta charset="UTF-8"><title>for 문</title></head>
<body>
<script>
  // 기본 for 문: 1부터 5까지 출력
  for (let i = 1; i ≤ 5; i++) {
    console.log(`${i}번째 반복`);
  }

  // 누적 합산
  let sum = 0;
  for (let i = 1; i ≤ 100; i++) {
    sum += i;
  }
  console.log('1~100 합계:', sum);    // 5050

  // 배열과 for 문
  const fruits = ['사과', '배', '딸기', '수박'];
  for (let i = 0; i < fruits.length; i++) {
    console.log(`${i}: ${fruits[i]}`);
  }
</script>
</body>
</html>
```

```
// 콘솔 출력:
1번째 반복
2번째 반복
3번째 반복
4번째 반복
5번째 반복
1~100 합계: 5050
0: 사과
1: 배
2: 딸기
3: 수박
```

줄별 코드 설명

줄	코드	설명
3	<code>for (let i = 1; i ≤ 5; i++)</code>	초기화 <code>let i = 1</code> , 조건 <code>i ≤ 5</code> , 증감 <code>i++</code> 을 세미콜론으로 구분합니다
4	<code>`\${i}번째 반복`</code>	템플릿 리터럴로 변수를 문자열에 삽입합니다
9	<code>sum += i</code>	<code>sum = sum + i</code> 의 축약 표현입니다. 매 반복마다 <code>i</code> 값을 누적합니다
15	<code>fruits.length</code>	배열의 요소 개수를 반환합니다. 배열이 바뀌어도 자동으로 맞춰집니다

`while`, `for ... of`, `break`, `continue` 를 종합적으로 보여줍니다.


```

<!DOCTYPE html>
<html lang="ko">
<head><meta charset="UTF-8"><title>while / for ... of / break / continue</title>
<body>
<script>
  // while 문
  let count = 0;
  while (count < 3) {
    console.log(`while 반복 ${count}`);
    count++; // 반드시 증가시켜야 무한 루프 방지
  }

  // do ... while: 최소 1회 실행
  let n = 10;
  do {
    console.log(`do-while 실행: n = ${n}`);
    n++;
  } while (n < 5); // 조건이 false지만 이미 한 번 실행됨

  // for ... of: 배열 순회
  const colors = ['red', 'green', 'blue'];
  for (const color of colors) {
    console.log(color); // 인덱스 없이 값만 꺼냄
  }

  // for ... of: 문자열 순회
  for (const ch of 'Hello') {
    process.stdout && process.stdout.write(ch + ' '); // 브라우저에서는 console.log
  }
  // 브라우저 콘솔: H e l l o 각각 출력
  for (const ch of 'Hi') {
    console.log(ch);
  }

  // break: 특정 조건에서 반복 종료
  for (let i = 0; i < 10; i++) {
    if (i === 5) break;
    console.log(`break 전: ${i}`);
  }

  // continue: 특정 조건을 건너뛸
  for (let i = 0; i < 6; i++) {
    if (i % 2 === 0) continue; // 짝수는 건너뛸
    console.log(`홀수: ${i}`);
  }
</script>

```

```
</body>
</html>
```

```
// 콘솔 출력:
while 반복 0
while 반복 1
while 반복 2
do-while 실행: n = 10
red
green
blue
H
i
break 전: 0
break 전: 1
break 전: 2
break 전: 3
break 전: 4
출수: 1
출수: 3
출수: 5
```

줄별 코드 설명

줄	코드	설명
3	<code>while (count < 3)</code>	<code>count</code> 가 3 보다 작은 동안 반복합니다. <code>count++</code> 로 매 반복마다 값을 증가시킵니다
10	<code>do { ... } while (n < 5)</code>	본문을 먼저 실행합니다. <code>n = 10</code> 이므로 조건이 <code>false</code> 지만 한 번은 출력됩니다
17	<code>for (const color of colors)</code>	<code>of</code> 키워드를 사용하여 배열 요소를 순서대로 꺼냅니다. <code>const</code> 로 선언하여 재할당을 방지합니다
28	<code>if (i === 5) break</code>	<code>i</code> 가 5 가 되는 순간 반복문을 완전히 종료합니다
33	<code>if (i % 2 === 0) continue</code>	<code>i</code> 가 짝수 (<code>i % 2 === 0</code>)이면 나머지 코드를 건너뛰고 다음 반복으로 이동합니다

베스트 프랙티스: 배열을 단순히 순회할 때는 `for ... of` 를 사용하세요. 인덱스가 필요할 때만 `for` 문을 사용합니다. 더 나아가 10장에서 배울 `forEach`, `map`, `filter`

같은 고차 메서드를 사용하면 더욱 선언적이고 읽기 쉬운 코드를 작성할 수 있습니다.

주의: `for ... of` 는 배열과 문자열 등 이터러블(iterable) 객체에 사용합니다. 일반 객체의 속성을 순회할 때는 `for ... in` 을 사용하지만, 프로토타입 체인의 속성까지 포함될 수 있어 주의가 필요합니다. 일반 객체 순회에는 `Object.keys()` , `Object.values()` 를 권장합니다.

절 요약

- `for (초기화; 조건; 증감)` 문은 반복 횟수를 알 때 사용합니다.
- `while` 은 조건 먼저 평가, `do ... while` 은 본문 먼저 실행합니다.
- `for ... of` 는 배열/문자열 등 이터러블을 간결하게 순회합니다.
- `break` 는 반복문 즉시 종료, `continue` 는 현재 반복만 건너뜁니다.
- `while` 사용 시 종료 조건을 반드시 확인하여 무한 루프를 방지합니다.

FAQ

Q1: `const` 로 선언했는데 왜 배열 요소를 추가할 수 있나요?

`const` 는 변수가 가리키는 주소(참조)를 바꾸지 못하도록 막습니다. 배열과 객체는 참조 타입이므로, 변수에는 배열 자체가 아닌 배열이 있는 메모리 주소가 저장됩니다. `push()` 나 속성 변경은 주소(참조)를 바꾸는 것이 아니라 그 주소가 가리키는 내용을 바꾸는 것이므로 허용됩니다. `const arr = [1, 2]; arr.push(3);` 은 정상 동작하지만, `arr = [4, 5];` 는 `TypeError` 를 발생시킵니다.

Q2: `'5' + 3` 이 `8` 이 아니라 `'53'` 이 되는 이유가 무엇인가요?

JavaScript의 `+` 연산자는 피연산자 중 하나라도 문자열이면 나머지를 문자열로 변환하여 **이어붙이기(concatenation)**를 수행합니다. `'5'` (문자열)와 `3` (숫자)이 만나면 `3` 이 `'3'` 으로 변환되어 `'5' + '3' = '53'` 이 됩니다. 숫자 연산을 원한다면 `Number('5') + 3 = 8` 과 같이 명시적으로 타입을 변환하세요.

Q3: `switch` 문에서 `break` 를 빠뜨리면 어떤 일이 생기나요?

`break` 없이 `case` 가 일치하면 그 아래의 모든 `case` 가 연속으로 실행됩니다(폴스루, fall-through). 예를 들어 `case 'A':` 다음에 `break` 없이 `case 'B':` 코드가 있으면, 'A' 조건에서 진입해도 'B' 코드까지 실행됩니다. 이것이 의도한 동작(여러 케이스가 같은 코드를 공유)이 아니라면 반드시 `break` 를 붙이세요.

Q4: `for ... of` 와 `for ... in` 의 차이는 무엇인가요?

`for ... of` 는 배열, 문자열, `Map`, `Set` 등 이터러블 객체의 값(value)을 순서대로 꺼냅니다. `for ... in` 은 객체의 열거 가능한 속성 키(key)를 순회하며, 프로토타입 체인의 속성까지 포함될 수 있습니다. 배열 순회에는 `for ... of` 를, 일반 객체 속성 순회에는 `Object.keys()` 나 `Object.entries()` 를 권장합니다.

Q5: `undefined` 와 `null` 을 어떻게 구분해서 사용해야 하나요?

`undefined` 는 시스템이 자동으로 부여하는 값입니다. 변수 선언 후 초기화 전, 함수에 인자를 전달하지 않았을 때, 존재하지 않는 객체 속성에 접근했을 때 등이 해당됩니다. `null` 은 개발자가 의도적으로 값이 없음을 표현할 때 직접 할당합니다. 예를 들어 API에서 아직 데이터를 받지 못한 상태를 나타낼 때 `let data = null;` 로 선언합니다.

장 요약

이번 장에서는 JavaScript를 HTML 페이지에 연결하는 방법부터 시작하여, 값을 저장하는 `const` / `let` 변수 선언, 데이터의 종류를 나타내는 자료형과 타입 변환, 그리고 비교 연산자와 조건문/반복문으로 프로그램 흐름을 제어하는 방법을 배웠습니다. 특히 `==` vs `===`, 암묵적 타입 변환, `var` 의 호이스팅 문제처럼 초보자가 자주 만나는 함정을 내부 동작 원리와 함께 살펴보았습니다. 이 기초 위에서 다음 장의 함수, 배열, 객체 학습이 훨씬 수월해질 것입니다.

핵심 키워드

`script`, `defer`, `console.log`, `const`, `let`, `var`, 블록 스코프, 호이스팅, `number`, `string`, `boolean`, `null`, `undefined`, `typeof`, 타입 변환, `===`, `==`, `if/else`, `switch`, 삼항 연산자, `for`, `while`, `for ... of`, `break`, `continue`

다음 장 미리보기

다음 장에서는 코드를 재사용 가능한 단위로 묶는 **함수(Function)**와, 여러 값을 구조화하는 **배열**과 **객체**, 그리고 `map` / `filter` / `reduce` 같은 강력한 고차 배열 메서드를 배웁니다.

🔗 실습 프로젝트 (Hands-on Project)

9장 실습 프로젝트: 숫자 맞추기 게임

이 장에서 배운 변수, 자료형, 연산자, 조건문, 반복문을 실무 시나리오에 적용하는 프로젝트입니다.

초급: 콘솔 숫자 맞추기 게임

시나리오

온라인 교육 플랫폼에서 학습 흥미를 높이기 위한 미니 게임을 만들어야 합니다. 1~100 사이의 숫자를 무작위로 정하고, 플레이어가 추측한 숫자와 비교하여 "더 높게", "더 낮게", "정답!" 힌트를 콘솔에 출력합니다. 최대 시도 횟수(10회)를 초과하면 게임이 종료됩니다.

학습 목표

- `const` 와 `let` 으로 게임 상태 변수를 관리합니다.
- `Math.random()` 과 `Math.floor()` 로 무작위 숫자를 생성합니다.
- `if/else if/else` 로 힌트 메시지를 결정합니다.
- `while` 반복문으로 시도 횟수를 제어합니다.

진행 방법

1. `project_starter.html` 을 열고 TODO 주석을 찾습니다.
2. 각 TODO를 이 장에서 배운 개념으로 완성합니다.
3. 완성 후 브라우저에서 열고 "게임 시작" 버튼을 클릭합니다.
4. 숫자를 입력하고 "추측하기" 버튼으로 힌트를 확인합니다.

실행 방법

브라우저에서 `project_starter.html` 파일을 열어 실행합니다.

중급: 고급 숫자 맞추기 게임 (통계 + 난이도)

시나리오

기본 숫자 맞추기 게임을 발전시켜, 난이도별로 숫자 범위와 최대 시도 횟수를 다르게 설정하고, 게임 종료 후 시도 횟수·최단 기록·평균 시도 횟수 등 통계를 관리하는 기능을 추가합니다. 게임 히스토리를 배열에 누적하고 `for...of`로 순회하여 결과를 표시합니다.

학습 목표

- 객체로 게임 설정(난이도, 범위, 최대 시도)을 구조화합니다.
- 배열에 게임 결과를 누적하고 `for...of`로 순회합니다.
- 반복문과 조건문을 조합하여 게임 상태 머신을 구현합니다.
- `??` (널 병합)와 `===`를 실무 패턴으로 활용합니다.

진행 방법

1. `project_advanced_starter.html`을 열고 TODO 주석을 찾습니다.
2. 초급에서 연습한 내용을 바탕으로 더 복잡한 로직을 완성합니다.
3. 완성 후 여러 번 게임을 플레이하여 통계가 올바르게 누적되는지 확인합니다.

실행 방법

브라우저에서 `project_advanced_starter.html` 파일을 열어 실행합니다.

확장 아이디어

- 최고 기록(최소 시도)을 `localStorage`에 저장하여 페이지를 새로고침해도 유지합니다.
- 게임 시간을 측정하여 빠른 정답에 보너스 점수를 부여합니다.
- 이전 추측 목록을 화면에 표시하여 플레이어가 힌트를 추적할 수 있게 합니다.

▶  project_starter.html (스타터 코드 - 직접 완성해보세요)

▶  project_complete.html (완성 코드)

▶  project_advanced_starter.html (스타터 코드 - 직접 완성해보세요)

▶  project_advanced_complete.html (완성 코드)